

Paradigmi di Programmazione - A.A. 2023-24

Esame Scritto del 9/9/2024

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione usando la strategia **call-by-name** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$((\lambda x. \lambda y. xy)((\lambda k. k)y))z$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f h x y =  
  if h x then match x with  
    | (0,k) -> k  
    | (_,k) -> h x  
  else y ;;
```

Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive interi “colorati” di nero o di rosso:

```
type colored_int =  
| Black of int      (* intero nero *)  
| Red of int        (* intero rosso *)
```

si definiscano, usando i costrutti di programmazione funzionale di OCaml, le funzioni `swap_colors` e `seq_len` con tipi

```
swap_colors : colored_int list -> colored_int list  
seq_len : colored_int list -> int
```

tali che:

- `swap_colors lis` restituisca una lista analoga a `lis` ma in cui gli interi neri sono rossi e viceversa;
- `seq_len lis` restituisca la lunghezza della più lunga sequenza di interi dello stesso colore consecutivi in `lis`

Ad esempio, data la seguente lista di interi colorati

```
let lis = [Black 10; Red 5; Red 2; Black 4; Black 1; Black 7; Red 2]
```

abbiamo che

```
swap_colors lis;; (* restituisce [Red 10; Black 5; Black 2; Red 4; Red 1; Red 7; Black 2] *)
```

```
seq_len lis;;      (* restituisce 3 -- in quanto ci sono tre interi neri in sequenza *)
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con un costrutto `Test(f;g;x;y)` che, date due espressioni che descrivono funzioni (non ricorsive) `f` e `g`, un'espressione che descrive un argomento `x` e un'espressione che descrive un possibile risultato `y`, verifichi se l'applicazione di `f` ad `x` dia come risultato `y`. In caso affermativo, il risultato di `Test` sarà `y`, altrimenti il risultato di `Test` dovrà essere ottenuto applicando `g` ad `x`.

NOTA: non usare il costrutto `Apply` nell'implementazione di `Test`.

Alcuni esempi (in una sintassi concreta simile a OCaml):

```
Test ( fun x -> x+1 ; fun x -> x-1 ; 10 ; 11);;  (* risultato = 11 *)
Test ( fun x -> x+1 ; fun x -> x-1 ; 10 ; 15);;  (* risultato = 9 *)
```

Altro esempio:

```
let f = fun x -> x+1 in
let g = fun x -> x-1 in
let x = 10 in
Test ( f ; g ; x ; x+1);;          (* risultato 11 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.